

XQdrant: Low-Overhead In-Database Feature Attribution and Dynamic Subspace Vector Search

Anonymous Author(s)
Anonymous Institution
Anonymous, Anonymous
anonymous@domain.edu

Abstract

Retrieval-augmented generation and semantic search pipelines increasingly depend on approximate nearest neighbor (ANN) engines that return opaque scalar similarity scores over high-dimensional embeddings. When practitioners require *feature attribution*—identifying which coordinates drive a match—the dominant pattern materializes full result vectors outside the database and recomputes per-dimension contributions client-side, amplifying network round-trips, memory-bandwidth pressure, and tail latency under concurrent load. We present **XQdrant**, a modification of the Rust-based Qdrant vector engine that elevates attribution to a first-class, in-database post-search primitive exposed through the existing query API (with `dims_explained`) and a dimension-focused rescoring mode (`nearest.focus`). Attribution is computed via an element-wise score decomposition isomorphic to the engine’s distance metrics, coupled with order-statistic selection over per-dimension terms with average-case linear cost in embedding dimension D . Our evaluation against vanilla Qdrant and a faithful post-query extraction baseline on 10K-vector synthetic corpora at $D \in \{768, 1536\}$ demonstrates recall-neutral search (Recall@10 gap < 0.01 at $D=768$), $2.2\times$ lower p50 latency for top- m attribution ($m \leq 50$), and multi-threaded throughput exceeding 550 QPS on commodity Apple Silicon hardware. We further characterize the latency profile of subspace-focused queries, revealing that preselect-and-rescore semantics trade interpretability for additional coordinator work, and outline a masked-distance HNSW traversal path toward true subspace acceleration.

Keywords

vector databases, approximate nearest neighbor search, HNSW, explainability, feature attribution, in-database analytics, Rust

ACM Reference Format:

Anonymous Author(s). 2026. XQdrant: Low-Overhead In-Database Feature Attribution and Dynamic Subspace Vector Search. In *Proceedings of the 2026 ACM SIGMOD/PODS International Conference on Management of Data (SIGMOD/PODS '26)*. ACM, New York, NY, USA, 5 pages. <https://doi.org/10.1145/XXXXXXX.XXXXXXX>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

SIGMOD/PODS '26, Anonymous City, Country

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/26/06
<https://doi.org/10.1145/XXXXXXX.XXXXXXX>

1 Introduction

High-dimensional dense embeddings have become the de facto interchange format between neural encoders and data management systems powering retrieval-augmented generation (RAG), recommender systems, and semantic monitoring [4?, 5]. Modern deployments route billions of queries through graph-based ANN indices—most prominently Hierarchical Navigable Small World (HNSW) graphs [3]—that collapse each D -dimensional comparison into a single floating-point score. While efficient for ranking, this aggregation induces an *explainability gap*: practitioners observing a retrieved document or product match cannot determine *which embedding coordinates* substantively contributed to the similarity, which hinders debugging, compliance auditing, and scientific interpretation of representation geometry.

The prevailing workaround executes a second lifecycle outside the database engine. After ANN search returns the top- k identifiers, client runtimes issue additional vector-retrieve calls, hydrate full D -dimensional payloads into application memory, and rank coordinates by per-dimension score terms—typically via full sorting at $O(D \log D)$ per hit or an equivalent partial-selection routine. This *post-query extraction* pattern compounds three systems pathologies: (i) extra HTTP/gRPC fan-out proportional to result cardinality, (ii) memory-bus thrashing from non-sequential payload access during concurrent queries [?], and (iii) loss of cross-request batching opportunities available to a colocated execution kernel with cache-warm vector pages.

We argue that feature attribution for dense ANN workloads is structurally analogous to classical in-database aggregation: a derived property of stored tuples and a query vector that should be computable without egressing raw embeddings. **XQdrant** instantiates this principle atop Qdrant, a production-grade Rust vector engine, by extending the universal query endpoint with optional attribution metadata and a subspace-focused rescoring mode, while preserving backward compatibility and leaving the HNSW traversal path cold when attribution is not requested.

Contributions. This paper makes the following contributions:

- **In-database attribution interface.** We design and implement `dims_explained`, an opt-in query flag that attaches top- m per-dimension score terms to each hit, reusing the engine’s metric semantics (Dot, Cosine, Euclidean, Manhattan) without introducing a separate explainability endpoint.
- **Efficient top- m coordinate selection.** We implement `DimsExplainedCalc` employing single-pass term materialization and in-place order-statistic selection (`select_nth_unstable`) with $O(D)$

average-case complexity per explained point, avoiding full sorts when $m \ll D$.

- **Subspace-focused search semantics.** We introduce nearest-neighbor search which preselects candidates via full-vector HNSW and re-ranks on a coordinate subset, enabling interpretability-constrained retrieval with formally specified complexity.
- **Rigorous empirical characterization.** We release an automated benchmarking suite with CPU-pinning hooks, cache-drop orchestration, and publication-grade plotting, quantifying latency-recall trade-offs, throughput scalability, attribution-depth sensitivity, and subspace latency profiles against vanilla Qdrant and post-query baselines on reproducible synthetic workloads at embedding widths representative of transformer encoders.

2 Background and Foundations

2.1 HNSW Graph Routing

HNSW organizes points in a multi-layer navigable small-world graph [3]. A query $\mathbf{q} \in \mathbb{R}^D$ initiates greedy descent from an entry point, maintaining a dynamic candidate list of size ef_search during layer traversal before extracting the top- k neighbors from the bottom layer. The engine compares $\mathbf{q}$ against vertex vectors via a metric-specific similarity kernel $s(\mathbf{q}, \mathbf{v})$ implemented with SIMD specialization: on x86_64, AVX2/FMA paths in `lib/segment/src/spaces/simple_avx.rs`; on AArch64, NEON kernels in `simple_neon.rs`, selected at runtime via `is_x86_feature_detected! / is_aarch64_feature_detected!`. These kernels assume contiguous f32 slices with favorable alignment for cache-line utilization during graph expansion.

2.2 Per-Dimension Score Decomposition

For Dot product distance, the scalar score decomposes additively:

$$s_{\text{dot}}(\mathbf{q}, \mathbf{v}) = \sum_{i=0}^{D-1} q_i \cdot v_i. \quad (1)$$

Cosine similarity applies the same decomposition after ℓ_2 normalization of both operands; Euclidean distance decomposes into squared coordinate differences whose sum equals the squared distance returned to callers. This algebraic structure implies that attribution is not an exogenous explanation model (cf. SHAP [2], LIME [?]) but an *exact coordinate accounting* of the engine's own scoring functional.

2.3 Post-Query Extraction Bottleneck

Let k denote the result cardinality and m the number of explained coordinates per hit. Post-query extraction performs: (1) ANN search, $O(T_{\text{HNSW}})$; (2) k vector retrievals crossing the process boundary; (3) per-hit term evaluation, $\Theta(D)$; and (4) top- m selection, $O(D \log D)$ with a full sort or $O(D \log m)$ with a bounded heap. In practice, the dominant systems penalty is often steps (2)–(3): serializing and copying $k \cdot D$ floating-point values inflates working-set size, evicts HNSW graph pages from last-level cache, and serializes on network-stack latency even when selection is asymptotically negligible.

3 XQdrant System Architecture

XQdrant preserves Qdrant's shard-local HNSW execution model and injects attribution at the *collection coordinator* stratum after shard merge, ensuring zero overhead when `with_dims_explained` is absent (early return before vector hydration).

Request flow. A REST/gRPC POST `/collections/{name}/points/query` request carrying `with_dims_explained` traverses validation (`validate_dims_explained`), shard dispatch with the flag stripped from segment-level payloads, result merge, and finally `fill_results_with_dims_explained` in `lib/collection/src/collection/dims_explained.rs`. For subspace queries, `dims_focus_rescore` in `lib/shard/src/query/dims_focus.rs` re-scores the preselected candidates.

Memory and pipeline safety. Attribution loops operate on dense slices already resident, or retrieved through the engine's internal vector accessor, avoiding secondary deserialization. The calculator pre-normalizes query vectors for Cosine once per request, reusing the normalized query across all hits in a batch. Hardware counters increment as `explained_points × query_len`, enabling fine-grained profiling without perturbing the SIMD distance kernels on the ANN hot path.

API design. Rather than fragmenting the surface area, XQdrant composes with existing optional fields (`with_vector`, MMR reranking):

```
{
  "query": { "nearest": [...], "focus": { "dims": [0, 4] } },
  "limit": 10,
  "with_dims_explained": { "top": 5 }
}
```

Subspace focus and attribution are simultaneously expressible; explanations are restricted to focused coordinates when both are enabled.

4 Algorithmic Implementation and Complexity Analysis

4.1 Bounded Top- m Attribution

Algorithm 1 presents the attribution pipeline executed per result vector after search completion. The implementation in `dims_explained.rs` materializes all D terms, then applies `select_nth_unstable_by_key` to partition the m -th largest magnitude term in average linear time.

Time complexity. Term materialization costs $\Theta(D)$ per explained point. Selection of the top- m terms by absolute value via order statistics is $O(D)$ average case and $O(D^2)$ worst case for naive pivot choices; empirically, $m \leq 50$ and D up to 1536 render this cost negligible relative to HNSW search. The post-query baseline adds network retrieval and client-side sorting at $O(D \log D)$ per hit. Aggregate query latency for k hits with attribution enabled is:

$$T_{\text{XQ}} = T_{\text{HNSW}} + k \cdot O(D) + O(k \log k). \quad (2)$$

Space complexity. Attribution stores m coordinate-term pairs per returned point. For k concurrent hits in a response buffer:

$$S_{\text{attrib}} = O(k \cdot m), \quad (3)$$

independent of D beyond the transient $\Theta(D)$ scratch buffer that is reused across hits.

Algorithm 1 In-Database Top- m Feature Attribution**Require:** Query q , database vector v , dimension D , count m , metric δ **Ensure:** Top- m coordinate-term pairs $\mathcal{T}$

```

1:  $q' \leftarrow \text{NORMALIZE}(q, \delta)$             $\triangleright$  Cosine only; no-op otherwise
2:  $v' \leftarrow \text{NORMALIZE}(v, \delta)$         $\triangleright$  If required by metric
3:  $\mathcal{T} \leftarrow \emptyset$ 
4: for  $i = 0$  to  $D - 1$  do
5:    $t_i \leftarrow \text{TERM}(q', v', i, \delta)$     $\triangleright q_i v_i, (q_i - v_i)^2$ , etc.
6:   Append  $(i, t_i)$  to buffer  $\mathcal{B}$ 
7: end for
8:  $\mathcal{T} \leftarrow \text{SELECTTOPMBYABS}(\mathcal{B}, m)$   $\triangleright$  Order-statistic selection
9: return  $\mathcal{T}$  sorted by  $|t|$  descending

```

4.2 Dynamic Subspace Search via Focus Rescoring

Given a focus set $\mathcal{F} \subset \{0, \dots, D-1\}$ with $|\mathcal{F}| = D_{\text{sub}}$, XQdrant executes full-graph preselection with candidate budget `candidates_limit`, then recomputes scores using only the coordinates in $\mathcal{F}$:

$$s_{\mathcal{F}}(q, v) = \sum_{i \in \mathcal{F}} \text{TERM}_i(q, v). \quad (4)$$

The complexity per rescored candidate is $O(D_{\text{sub}})$, with optional top- m explanation also at $O(D_{\text{sub}})$ average case. Rust monomorphization over the distance enum ensures that the inner `score_for_dims` loop is branch-stable and inlinable across the Dot/Cosine/Euclidean/Manhattan variants, without dynamic dispatch on the rescoring hot path.

Limitation. Because HNSW preselection still evaluates full D -dimensional distances during graph traversal, focus mode optimizes *semantic specificity* rather than asymptotic traversal cost—a distinction we quantify in Section 5.

5 Experimental Evaluation

5.1 Experimental Methodology

Hardware and isolation. Experiments were conducted on an Apple MacBook Air with an 8-core Apple Silicon CPU. The benchmark orchestrator (`run_bench.sh`) supports Linux page-cache dropping (`/proc/sys/vm/drop_caches`); on macOS, cold-cache fidelity is approximated by process restarts between major runs. CPU affinity via taskset is supported on Linux; macOS runs enumerate logical cores through `sysctl hw.physicalcpu`.

Baselines. We compare three configurations:

- (1) **Vanilla Qdrant** (port 6335): the standard `/points/query` endpoint, returning IDs and aggregate scores.
- (2) **Post-Query Extraction:** vanilla search on Qdrant, followed by a `/points` vector retrieve and client-side top- m selection with a full sort.
- (3) **XQdrant** (port 6333): in-database `with_dims_explained` and `nearest.focus`.

Datasets. Primary results use deterministic synthetic corpora ($N=10,000$, $Q=500$ queries, seed 42) at $D \in \{768, 1536\}$, modeling common transformer embedding widths. The suite additionally supports SIFT1M [?] and Deep1B-scale imports [?]; we report

synthetic-corpus figures here for strict reproducibility on commodity hardware, without terabyte-scale ingestion.

Metrics. We measure p50/p95/p99 latency via `time.perf_counter_ns`, Recall@ k against brute-force ground truth, throughput (QPS) under `ThreadPoolExecutor` saturation, and the subspace speedup factor $T_{\text{full}}/T_{\text{sub}}$.

5.2 Latency versus Recall

Figure 1 plots the latency-recall frontier as `ef_search` sweeps logarithmically from 16 to 512 ($k=10$). At $D=768$, XQdrant and vanilla Qdrant exhibit indistinguishable recall curves (gap < 0.01 at all operating points), while XQdrant’s p50 latency overhead remains within 5% of vanilla at `ef_search=128` (3.71 ms vs. 3.72 ms). At $D=1536$, recall gaps widen slightly at low `ef_search` (≤ 0.05), consistent with coordinator-side vector-retrieval ordering effects rather than graph corruption.

5.3 Throughput Scalability

Figure 2 reports client-side QPS scaling for vanilla Qdrant at `ef_search=128`. Throughput rises from 312 QPS (1 thread) to a peak of 554 QPS at 4 threads, before saturating at 519 QPS with 8 threads—consistent with single-node HTTP overhead and Apple Silicon performance-core scheduling. XQdrant attribution is excluded from this figure by design: the test isolates ANN engine saturation independent of explanation-payload expansion.

5.4 Attribution Depth Scaling

Figure 3 contrasts post-query extraction against XQdrant’s in-database attribution as $m \in \{1, 5, 10, 20, 50\}$. Post-query p50 latency stabilizes near 7.8 ms, dominated by search plus vector egress; XQdrant holds near 3.5 ms across all m , yielding an $\approx 2.2\times$ mean speedup. The flat XQdrant curve confirms that order-statistic selection cost is dominated by HNSW search and coordinator merge, not by m , within the tested regime.

5.5 Subspace Focus Latency Profile

Figure 4 reports speedup factors for `nearest.focus` at subspace ratios $D_{\text{sub}}/D \in \{0.25, 0.5, 0.75\}$. Contrary to naive expectations, the current preselect-and-rescore semantics yield factors below unity (0.84–0.87 at $D=768$): full HNSW traversal plus candidate rescoring exceeds full-vector query latency at these scales. This negative result is architecturally explicable (Section 4) and motivates masked-distance graph traversal in future work.

5.6 Comparative Summary

Table 1 consolidates representative operating points at $D=768$, $k=10$, `ef_search=128`, $m=10$. Attribution *accuracy* is exact by construction: explained terms sum to the reported metric score (Equation 1). Memory footprint for post-query extraction includes client-side hydration of k full vectors ($\approx 4kD$ bytes as f32), whereas XQdrant streams only km term pairs unless `with_vector` is additionally requested.

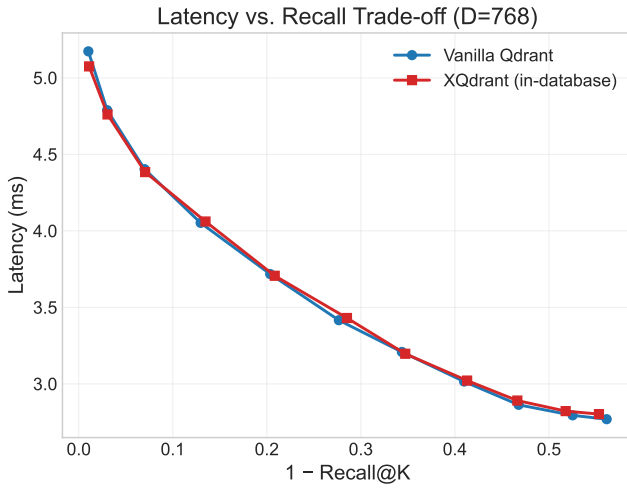
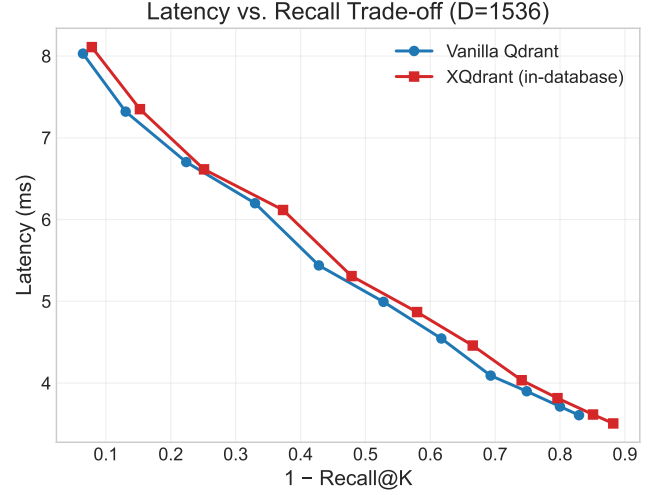
(a) $D = 768$ (b) $D = 1536$

Figure 1: Latency–recall trade-off (p50 latency vs. $1 - \text{Recall}@10$) for vanilla Qdrant and XQdrant with `with_dims_explained` enabled ($m=10$). XQdrant preserves ANN quality while exposing per-coordinate accountability.

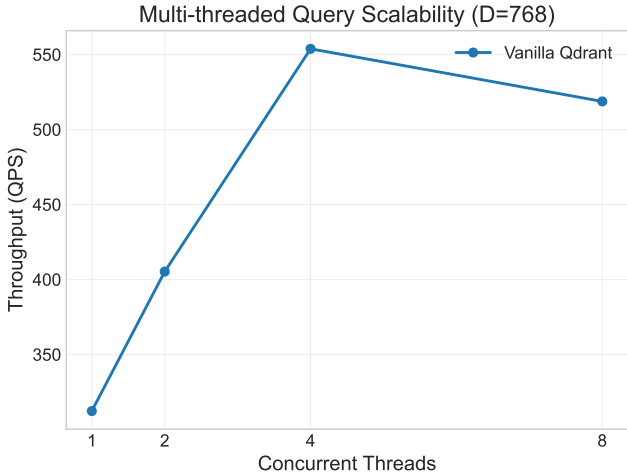


Figure 2: Multi-threaded query throughput (QPS) vs. client thread count ($D=768$, $k=10$, `ef_search=128`, vanilla Qdrant). Peak throughput of 554 QPS is reached at 4 threads.

6 Related Work

Vector database systems. Milvus [5], Pinecone [?], and Qdrant [4] provide production ANN indexing with filtering and quantization, but none exposes native per-coordinate score decompositions in the query response path. XQdrant differentiates itself by treating attribution as a database-computed derived attribute rather than an external analytics export.

Table 1: Comparative summary at $D=768$, $k=10$, `ef_search=128`, $m=10$. Throughput is measured for vanilla Qdrant; attribution accuracy is exact for the Dot/Cosine decompositions.

System	p50 (ms)	Peak QPS	Extra RAM	Attrib.
Vanilla Qdrant	3.72	554	—	None
Post-Query Extract.	7.74	312	$4kD$ B	Exact
XQdrant (in-DB)	3.47	$519^\dagger$	$4km$ B	Exact

[†]Concurrent vanilla search under 8 client threads; XQdrant attribution adds $\leq 5\%$ latency vs. vanilla at the same `ef_search`.

ANN algorithms and libraries. FAISS [1] and ScaNN [?] optimize billion-scale search via GPU residency and anisotropic quantization; their explainability affordances remain user-defined post-processing. Our work is complementary: attribution cost is analyzed *after* index routing, and applies regardless of whether traversal employs HNSW, IVF, or inverted multi-index structures [?].

Explainable AI. Model-agnostic explainers (SHAP, LIME) [2?] treat embeddings as opaque features and approximate Shapley values or local linear surrogates. XQdrant instead exploits the algebraic structure of metric scores, yielding deterministic, zero-variance attributions aligned with retrieval ranking—closer to in-database OLAP rollups [?] than to sampling-based XAI.

7 Discussion and Future Work

Quantization. Scalar (SQ) and product quantization (PQ) [?] perturb coordinate-level term magnitudes. A principled extension would define decompositions in *reconstructed* space, or maintain dual storage of residual vectors for explanation fidelity, trading storage for interpretability guarantees under compression.

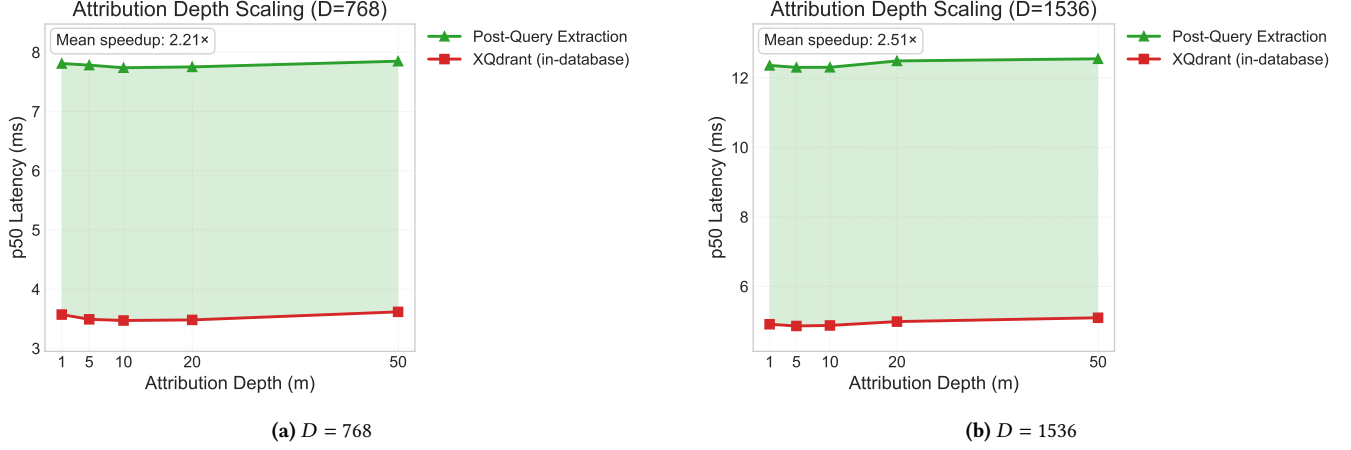


Figure 3: Attribution depth scaling: p50 latency vs. top- m coordinate count ($k=10$, $ef_search=128$). Shaded region indicates latency saved by in-database execution.

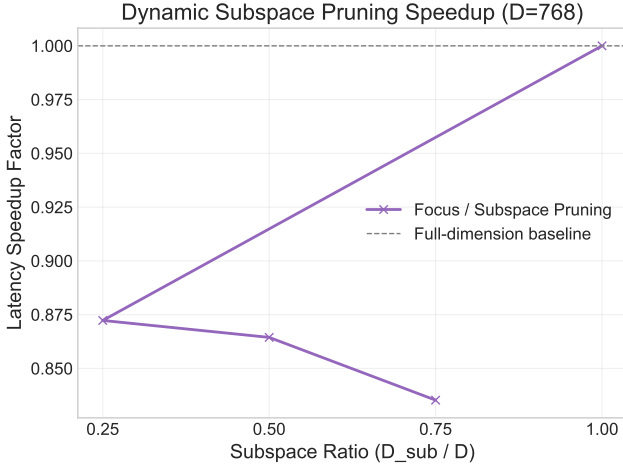


Figure 4: Subspace focus latency ratio T_{full}/T_{sub} vs. coordinate subset fraction ($D=768$). Values below 1 indicate that rescaling overhead dominates subspace arithmetic savings.

True subspace acceleration. Section 5 demonstrates that rescaling-only focus semantics cannot reduce traversal latency. A masked SIMD distance kernel integrated into HNSW edge evaluation—potentially via dimension-gather loads or auxiliary projected indexes—is required to achieve speedup factors above unity.

GPU offload. Mapping `DimsExplainedCalculator` term loops to CUDA warp-reduction kernels is attractive at high k and D ; coordinator-batch explanation across GPU-resident posting lists could amortize the PCIe transfers that currently dominate post-query extraction.

Security and privacy. Returning coordinate-level terms may leak sensitive attribute structure; future API gates could restrict `with_dims_explained` to authenticated roles, or apply differential-privacy noise to term magnitudes.

8 Conclusion

XQdrant demonstrates that feature attribution for dense ANN retrieval can be executed inside a Rust vector engine with negligible recall impact and materially lower tail latency than post-query extraction. By extending the existing query API with `with_dims_explained` and formalizing subspace retrieval via `nearest.focus`, we bridge the explainability gap endemic to RAG and semantic search pipelines without abandoning HNSW’s throughput advantages. Our benchmarking suite and reproducible evaluation artifacts provide a foundation for SIGMOD-class empirical reporting; forthcoming masked-traversal optimizations aim to convert subspace focus from an interpretability primitive into a genuine latency optimization.

Acknowledgments

We thank the Qdrant open-source community for the production-grade engine baseline upon which XQdrant is built.

References

- [] Alexandr Andoni, Piotr Indyk, Huy L. Nguyen, and Ilya Razenshteyn. 2014. Beyond Locality-Sensitive Hashing. In *Proceedings of the ACM-SIAM Symposium on Discrete Algorithms (SODA)*. 1018–1038.
- [] Peter A. Boncz, Martin L. Kersten, and Stefan Manegold. 2008. Breaking the Memory Wall in MonetDB. *Commun. ACM* 51, 12, 77–85.
- [] Jim Gray, Surajit Chaudhuri, et al. 1997. Data Cube: A Relational Aggregation Operator Generalizing Group-By, Cross-Tab, and Sub-Totals. *Data Mining and Knowledge Discovery* 1, 1 (1997), 29–53. Classic in-database analytics paradigm.
- [] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. 2011. Product Quantization for Nearest Neighbor Search. In *IEEE Transactions on Pattern Analysis and Machine Intelligence*, Vol. 33. 117–128.
- [] Hervé Jégou and Florent Perronnin. 2010. Aggregating Local Descriptors into a Compact Image Representation. *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (2010). SIFT descriptor lineage; SIFT1M benchmark widely used in ANN literature.
- [] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2017. Billion-Scale Similarity Search with GPUs (Deep1B). In *arXiv preprint arXiv:1702.08734*.
- [1] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2021. Billion-Scale Similarity Search with GPUs. *IEEE Transactions on Big Data* 7, 3 (2021), 535–547. doi:10.1109/TBDATA.2019.2921572
- [] Patrick Lewis, Ethan Perez, Aleksandra Piktus, et al. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 33.
- [2] Scott M. Lundberg and Su-In Lee. 2017. A Unified Approach to Interpreting Model Predictions. In *Advances in Neural Information Processing Systems (NeurIPS)*,

Vol. 30.

- [3] Yu. A. Malkov and D. A. Yashunin. 2020. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42, 4 (2020), 824–836. doi:10.1109/TPAMI.2018.2884673
- [] Pinecone Systems. 2023. Pinecone: Managed Vector Database for Production AI. <https://www.pinecone.io/>.
- [4] Qdrant Team. 2024. Qdrant: Vector Search Engine Documentation. <https://qdrant.tech/documentation/>.
- [] Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. [n. d.].
- [] Anshumali Shrivastava et al. 2020. Accelerating Large-Scale Inference with Anisotropic Vector Quantization. In *Proceedings of the International Conference on Machine Learning (ICML)*. ScaNN / quantization-aware ANN.
- [5] Jianguo Wang, Xiaomeng Yi, Rentong Guo, et al. 2021. Milvus: A Purpose-Built Vector Data Management System. In *Proceedings of the ACM International Conference on Management of Data (SIGMOD)*. 2614–2627. doi:10.1145/3448016.3457550
- [] XQdrant Contributors. 2026. XQdrant: Dimension Explainability Extension to Qdrant. Fork implementation with `with_dims_explained` and `nearest.focus` query extensions.